

ICS第一次月考10~11题讲评

王嘉怡 2400013107

试题讲解

第10题：

10. 对于以下汇编指令，给出与之内在操作流程相同、只是会修改目的寄存器的指令：

```
cmpq %rbx,%rax →  
testq %rbx,%rax →
```

基础知识考查题。

对应课本内容：

除了图 3-10 中的指令会设置条件码，还有两类指令（有 8、16、32 和 64 位形式），它们只设置条件码而不改变任何其他寄存器；如图 3-13 所示。CMP 指令根据两个操作数之差来设置条件码。除了只设置条件码而不更新目的寄存器之外，CMP 指令与 SUB 指令的行为是一样的。在 ATT 格式中，列出操作数的顺序是相反的，这使代码有点难读。如果两个操作数相等，这些指令会将零标志设置为 1，而其他的标志可以用来确定两个操作数之间的大小关系。TEST 指令的行为与 AND 指令一样，除了它们只设置条件码而不改变目的寄存器的值。

典型的用法是，两个操作数是一样的（例如，testq %rax,%rax 用来检查%rax 是负数、零，还是正数），或其中的一个操作数是一个掩码，用来指示哪些位应该被测试。因此（我的）答案是：

```
1  subq %rbx, %rax  
2  andq %rbx, %rax
```

指令	基于	描述
CMP S_1, S_2 cmpb cmpw cmpl cmpq	$S_2 - S_1$	比较 比较字节 比较字 比较双字 比较四字
TEST S_1, S_2 testb testw testl testq	$S_1 \& S_2$	测试 测试字节 测试字 测试双字 测试四字

图 3-13 比较和测试指令。这些指令不修改任何寄存器的值，只设置条件码

第11题：

11. 下面的 C 语言代码与汇编语言代码对应

<pre>int cmpxy(long x, long y) { return x > y; }</pre>	<pre>cmpq %rsi, %rdi setg %al movzbl %al, %eax ret</pre>
---	--

(1) 修改 C 语言代码如下，相应补全汇编语言代码

<pre>int cmpxy(long x, long y) { return x <= y; }</pre>	<pre>cmpq %rsi, %rdi _____ movzbl %al, %eax ret</pre>
--	--

(2) movzbl 指令本身的功能是：

(3) 在 x86-64 中，这条 movzbl 隐含操作还有：

阅读汇编代码：

```
1  cmpq    %rsi, %rdi    # 比较%rdi(x)和%rsi(y)，设置条件码
2  setg    %al           # 如果%rdi大于%rsi(~(SF^OF)&~ZF)，将%al设置为1，否则设置为0
3  movzbl  %al, %eax     # 将%al零扩展到%eax
4  ret
```

其中 %al, %eax, %rax 分别是返回值寄存器字节、双字和四字部分的名字。%rdi 是函数第一个参数寄存器，保存 x，%rsi 是函数第二个参数寄存器，保存 y。

(1) 对比两段代码。C代码中把 x>y 修改为 x<=y，汇编代码横线以外的部分都没有改变。因此汇编代码逻辑不变，我们只需要让 %al 在 x<=y 而不是 x>y 的时候为1。

回想 SET 类指令，此处应填写：

```
1  setle %al
```

(2) 基础知识考查题

对应课本内容：

指令	效果	描述
MOVZ S, R	R←零扩展(S)	以零扩展进行传送
movzwb		将做了零扩展的字节传送到字
movzbl		将做了零扩展的字节传送到双字
movzwl		将做了零扩展的字传送到双字
movzbq		将做了零扩展的字节传送到四字
movzwb		将做了零扩展的字传送到四字

因此 movzbl 指令的功能就是

- 1 将做了零扩展的字节传送到双字。

(3) 联想x86-64惯例:

任何为寄存器生成32位值的指令都会把该寄存器的高位部分置为0

因此这条 `movzbl` 指令的隐含操作是:

- 1 将%rax的高32位部分置零。

补充复习

关于条件码

CF进位(Carry), ZF零(Zero), SF符号(Sign), OF溢出(Overflow)标志

特殊情况:

1. `leaq` 不改变条件码
2. 逻辑操作中, `CF` 和 `OF` 会设置为 0
3. 移位操作中, `CF` 设置为最后一个被移除的位
4. `INC` 和 `DEC` 设置 `OF` 和 `ZF`, 不会改变 `CF`

关于题目中考查到的指令及注意事项

1. **传送指令**: `MOV`, `MOVS`, `MOVZ`
 1. 生成32位值的指令会把高位置零
 2. 源和目的不能同时为内存地址
 3. `movq` 是把32位补码数作为源操作数, 符号扩展到64位, 放到目的位置。传送64位立即数值需要用 `movabsq`, 并且只能以寄存器为目的。
 4. `cltq` 表示把 `%eax` 符号扩展到 `%rax` (`= movslq %eax, %rax`)
 5. 没有 `movzblq`, 因为等价于 `movzbl`。因此`MOVZ`类有五个指令, `MOVS`类有7个指令(没用的知识增加子)。
2. **设置条件码的指令**: `CMP`, `TEST`
 1. 只设置条件码不修改寄存器
 2. `CMP = AND`, `TEST = SUB`
 3. 常见用法: `testq %rax, %rax` 用来检查 `%rax` 是正数、零还是负数。
3. **访问条件码指令**: `SET`
 1. 名字很容易通过英文推断出来: not, equal, sign, great, less, above, below... (JMP类同理)
 2. 注意 `sete/setne` 也可以用于设置零/非零。

心得

基础知识扎实, 课本的细节不要放过, 考场上快速反应。